

Introducción C#

Módulo 1

Sintaxis y semántica del lenguaje C#

Variables y constantes

¿Qué es una variable?

Una **variable** representa una posición en memoria en el que se puede almacenar un valor de un determinado tipo al que se le da un cierto nombre. Para poder utilizarla sólo hay que definirla indicando cuál será su nombre y cuál será el tipo de datos que podrá almacenar, lo que se hace siguiendo la siguiente sintaxis:

<tipoVariable> <nombreVariable>;

Una variable puede ser definida dentro de una definición de clase, en cuyo caso se la denominaría **campo**. También puede definirse como una **variable local** a un

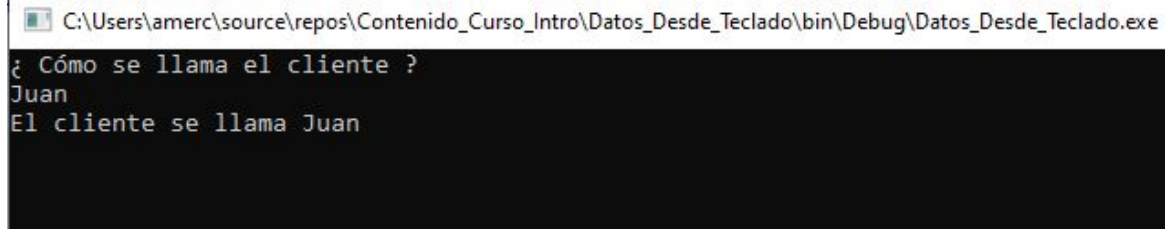
método, que es una variable definida dentro del código del método a la que sólo puede accederse desde dentro de dicho código. Otra posibilidad es definirla como **parámetro** de un método, que son variables que almacenan los valores de llamada al método y que, al igual que las variables locales, sólo pueden ser accedidas desde código ubicado dentro del método.

El siguiente ejemplo muestra cómo definir variables de todos estos casos:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Datos_Desde_Teclado
{
    class CapturaDatos
    {
        static void Main(string[] args)
        {
            string Nombre; // Declaración de la variable Nombre, de tipo string
            Console.WriteLine("¿ Cómo se llama el cliente ?");
            //Captura del valor ingresado y asignación de ese valor a Nombre, a través de Console.ReadLine();
            Nombre = Console.ReadLine();
            //Mostrar en la posición 0 la variable Nombre
            Console.WriteLine("El cliente se llama {0}", Nombre);
            Console.ReadKey();
        }
    }
}
```

En este sencillo programa, el compilador reservará memoria para la variable **Nombre**. En la ejecución del mismo, primero preguntará por el nombre del cliente y, después de haberlo escrito nosotros, al presionar Enter, nos dirá cómo se llama:



```
C:\Users\amerc\source\repos\Contenido_Curso_Intro\Datos_Desde_Teclado\bin\Debug\Datos_Desde_Teclado.exe
¿ Cómo se llama el cliente ?
Juan
El cliente se llama Juan
```

También podemos inicializar el valor de una variable en el momento de declararla, sin que esto suponga un obstáculo para poder modificarlo después:

```
int num = 10;
```

También tenemos las **constantes** que hacen que el compilador reserve un espacio de memoria para almacenar un dato, pero en este caso ese dato es siempre el mismo y no se puede modificar durante la ejecución del programa.

Un ejemplo claro sería almacenar el valor de **pi** en una constante para no tener que poner el número en todas las partes donde lo podamos necesitar. Otro ejemplo, la cantidad de días de la semana, la cantidad de meses del año, etc. Se declaran de un modo similar a las variables, basta con añadir la palabra **const** en la declaración.



Ejemplo:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Circunferencia
{
    class Perimetro
    {
        static void Main(string[] args)
        {
            const double PI = 3.1415926; // Declaración de constante PI, de tipo double
            double Radio = 4; // // Declaración e inicialización de constante Radio, de tipo double

            Console.WriteLine("El perímetro de una circunferencia de radio {0} es {1}", Radio, 2 * PI * Radio);

            Console.WriteLine("El área de un círculo de radio {0} es {1}", Radio, PI * Math.Pow(Radio, 2));
            Console.ReadKey();
        }
    }
}
```

La salida en la consola de este programa sería la siguiente:

```
C:\Users\amerc\source\repos\Contenido_Curso_Intro\Circunferencia\bin\Debug\Circunferencia.exe
El perímetro de una circunferencia de radio 4 es 25,1327408
El área de un círculo de radio 4 es 50,2654816
```



Tipos de datos

Introducción

El sistema de **tipos de datos** suele ser la parte más importante de cualquier lenguaje de programación. El uso correcto de los distintos tipos de datos es algo fundamental para que una aplicación sea eficiente con el menor consumo posible de recursos.

Actualmente, muchos de los lenguajes orientados a objetos proporcionan los tipos agrupándolos de dos formas: los **tipos primitivos del lenguaje**, como números o cadenas, y el resto de **tipos creados a partir de clases**. Esto genera muchas dificultades, ya que los tipos primitivos no son y no pueden tratarse como objetos, es decir, no se pueden

derivar y no tienen nada que ver unos con otros. Sin embargo, en C# (más propiamente en .NET Framework) contamos con un sistema de tipos unificado, el CTS (*Common Type System*), que proporciona todos los tipos de datos como clases derivadas de la clase de base *System.Object* (incluso los literales pueden tratarse como objetos). Sin embargo, el hacer que todos los datos que ha de manejar un programa sean objetos puede provocar que baje el rendimiento de la aplicación. Para solventar este problema, .NET Framework divide los tipos en dos grandes grupos: los **tipos valor** y los **tipos referencia**.

Cuando se declara una variable que es de un **tipo valor** se está reservando un espacio de memoria en la *pila* (stack) para que almacene los datos reales que contiene esta variable. Por ejemplo, en la declaración:

```
int num = 10;
```

Se está reservando un espacio de 32 bits en la pila (una variable de tipo int es un objeto de la clase System.Int32), en los que se almacena el 10, que es lo que vale la variable. Esto hace que la variable “num” se pueda tratar directamente como si fuera de un tipo primitivo en lugar de un objeto, mejorando notablemente el rendimiento. Como consecuencia, una variable de **tipo valor** nunca puede contener null (referencia nula).

Durante la ejecución de un programa, la memoria se distribuye en tres bloques: la *pila* (stack), el *montón* (heap) y la *memoria global*. La **pila** es una estructura en la que los elementos se van apilando, de modo que el último elemento en entrar en la pila es el primero en salir (estructura LIFO, o sea, *Last In First Out*). El **montón** es un bloque de memoria contiguo en el cual la memoria no se reserva en un orden determinado como en la pila, sino que se va reservando aleatoriamente según se va necesitando. Cuando el programa requiere un bloque del montón, este se sustrae y se retorna un puntero (variable que contiene direcciones de memoria) al principio del mismo.

La **memoria global** es el resto de memoria de la máquina que no está asignada ni a la pila ni al montón, y es donde se colocan el método main y las funciones que éste invocará.

En el caso de una variable que sea de un **tipo referencia**, lo que se reserva es un espacio de memoria en el montón para almacenar el valor, pero lo que se devuelve internamente es una referencia al objeto, es decir, un puntero a la dirección de memoria que se ha reservado. En este caso, una variable de un tipo referencia si puede contener una referencia nula (null).

¿Por qué esta “separación” de tipos y posiciones de memoria? Porque una variable de un tipo valor funcionará como un tipo primitivo siempre que sea necesario, pero podrá funcionar

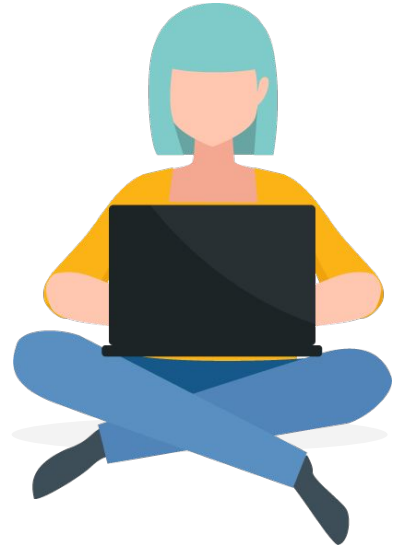
también como un tipo referencia, es decir como un objeto, cuando se necesite que sea un objeto. Hacer esto en otros lenguajes, como Java, es imposible, dado que los tipos primitivos en Java no son objetos. Más adelante ampliaremos estos conceptos.

Los **tipos de datos básicos** son ciertos tipos de datos tan comúnmente utilizados en la escritura de aplicaciones que en C# se ha incluido una sintaxis especial para tratarlos. Por ejemplo, para representar números enteros de 32 bits con signo se utiliza el tipo de dato **System.Int32** definido en la BCL, aunque a la hora de crear un objeto a de este tipo que represente el valor 2 se usa la siguiente sintaxis:

```
System.Int32 a = 2;
```

Dado lo frecuente que es el uso de este tipo también se ha predefinido en C# el alias **int** para el mismo, por lo que la definición de variable anterior queda así de compacta:

```
int a = 2;
```



System.Int32 no es el único tipo de dato básico incluido en C#. En el espacio de nombres **System** se han incluido todos estos:

Tipo	Descripción	Bits	Rango de valores	Alias
SByte	Bytes con signo	8	[-128, 127]	Sbyte
Byte	Bytes sin signo	8	[0, 255]	byte
Int16	Enteros cortos con signo	16	[-32.768, 32.767]	short
UInt16	Enteros cortos sin signo	16	[0, 65.535]	ushort
Int32	Enteros normales	32	[-2.147.483.648, 2.147.483.647]	int
UInt32	Enteros normales sin signo	32	[0, 4.294.967.295]	uint
Int64	Enteros largos	64	[-9.223.372.036.854.775.808, 9.223.372.036.854.775.807]	long
UInt64	Enteros largos sin signo	64	[0-18.446.744.073.709.551.615]	ulong
Single	Reales con 7 dígitos de precisión	32	[1,5×10-45 - 3,4×1038]	float
Double	Reales de 15-16 dígitos de precisión	64	[5,0×10-324 - 1,7×10308]	double
Decimal	Reales de 28-29 dígitos de precisión	128	[1,0×10-28 - 7,9×1028]	decimal
Boolean	Valores lógicos	32	true, false	bool
Char	Caracteres Unicode	16	['\u0000', '\uFFFF']	char
String	Cadenas de caracteres	Variable	El permitido por la memoria	string
Object	Cualquier objeto	Variable	Cualquier objeto	object

Tipos de datos básicos

Pese a su sintaxis especial, en C# los tipos básicos son tipos del mismo nivel que cualquier otro tipo del lenguaje. Es decir, heredan de **System.Object** y pueden ser tratados como objetos de dicha clase por cualquier método que espere un **System.Object**, lo que es muy útil para el diseño de rutinas genéricas que admitan parámetros de cualquier tipo y es una ventaja importante de C# frente a lenguajes similares como Java donde los tipos básicos no son considerados objetos.

El valor que por defecto se le da a los campos de tipos básicos consiste en poner a cero todo el área de memoria que ocupen. Esto se traduce en que los campos de tipos básicos

numéricos se inicializan por defecto con el valor 0, los de tipo **bool** lo hacen con **false**, los de tipo **char** con `'\u0000'` (carácter nulo), y los de tipo **string** y **object** con **null**.



Para los casos de los tipos de variables **numéricas** se puede especificar (durante la asignación de un valor), el *tipo* de dato numérico *de ese valor* mediante el uso de **sufijos**.

Los sufijos para los valores literales de los distintos tipos de datos numéricos son los siguientes:

- **L** (mayúscula o minúscula): *long* ó *ulong*, por este orden;
- **U** (mayúscula o minúscula): *int* ó *uint*, por este orden;
- **UL** ó **LU** (mayúsculas o minúsculas): *ulong*;
- **F** (mayúscula o minúscula): *single*;
- **D** (mayúscula o minúscula): *double*;
- **M** (mayúscula o minúscula): *decimal*;



¡Muchas gracias!

¡Sigamos trabajando!